

# **Teórico 9: XML y JSON**

# XML

- Estructura de datos XML.
- Esquemas de documentos XML.
- Consultas y transformaciones.
- APIs para XML.
- Almacenamiento de datos XML.



# Introducción

- XML: Extensible Markup Language.
- Definido por la WWW Consortium (W3C).
- Puede representar datos de base de datos como muchas clases de datos estructurados.
- Es muy utilizado para comunicar o integrar aplicaciones(esta siendo desplazado por el formato JSON).
- Los documentos tienen marcas dando información adicional sobre las secciones del documento.
  - Ej. `<titulo> XML </titulo> <transparencia> Introducción ...</ transparencia >`



# XML Introducción (Cont.)

La capacidad de especificar nuevas etiquetas, y de crear las estructuras anidadas hace a XML una técnica muy apropiada para intercambio de datos.

- Las marcas hacen datos (relativamente) autodocumentados.

- Ejemplo:

```
<supermercado>
  <articulo>
    <nro_articulo> 10 </ nro_articulo >
    <descripcion> Harina </descripcion >
    <precio> 20.8 </precio>
  </articulo>
  <proveedor>
    <nro_proveedor> 202 </nro_proveedor>
    <nombre> Molinos cañuelas </nombre>
  </proveedor>
</supermercado>
```



# XML: Motivación

- Hoy en día el intercambio de datos entre aplicaciones es crítico.
  - Ejemplos:
    - Bancos: transferencias de fondos entre cuentas de distintos bancos.
    - Procesamiento de ordenes entre varias empresas.
    - Lenguaje muy utilizado en Web Services, el protocolo SOAP está basado en XML.
- Cada aplicación tiene su propia forma de representar la información.
- XML se ha convertido en la base para todos los formatos de intercambio de datos de nueva generación.



# XML Motivación (Cont.)

- Fácil generación de formatos basados en texto plano con headers que indican el significado de los campos
- Cada tipo de documento basado en XML define cuáles son los elementos válidos, usando:
  - Tipos de lenguajes para especificar la sintaxis:
    - DTD (Document Type Descriptors).
    - XML Schema.
  - Se puede agregar una descripción textual de la semántica de una marca.
- XML permite definir nuevas marcas cuando se lo requiere, sin embargo esto puede ser restringido por el archivo de metadatos (DTDs o XML Schema).
- Una gran variedad de herramientas está disponible para analizar, mostrar y consultar documentos de XML/datos.



# Comparación con Datos Relacionales

- Ineficiente: Las marcas se repiten en cada fila de una tabla
- Mejor que datos relacionales a la hora intercambio de datos.

# Esquema de Documento XML

- Los esquemas de bases de datos restringen la información que pueden almacenar en cuanto a cantidad y tipo de datos.
- Los documentos XML no requieren tener asociado un esquema que defina su estructura.
- Sin embargo, los esquemas son muy importantes para el intercambio de datos XML.
- Existen 2 mecanismos para especificar esquemas XML
  - **Document Type Definition (DTD)**
    - Muy utilizado antes que surgiera XML Schema.
  - **XML Schema**
    - Más reciente, más utilizado en la actualidad.



# Document Type Definition (DTD)

- El tipo de un documento XML puede ser especificado utilizando DTD.
- Un DTD restringe la estructura de los datos XML o sea que elementos puede tener.
- Que atributos debe o puede tener un elemento.
- Que subelementos debe o puede tener incluido cada elemento y cuantas veces.
- Un DTD no restringe el tipo de datos, todos los valores son string en XML.
- Sintaxis DTD
  - `<!ELEMENT element (subelements-specification) >`
  - `<!ATTLIST element (attributes) >`



# XML Schema

- XML Schema es un lenguaje más sofisticado que fue el reemplazo de los DTDs. XML Schema soporta:
  - Tipos de datos.
  - Restricciones de valores mínimos y máximos.
  - Tipos de datos definidos por el usuario.
  - Además soporta claves, claves foráneas, herencia, etc.
- XML Schema se especifican en sintaxis XML,
- XML Schema está integrada con espacio de nombres.
- XML Schema más complejo que los DTDs.

# Otras características XML Schema

- Los atributos se especifican con la marca xs:attribute:
  - `<xs:attribute name = “numero_cuenta”/>`
  - Con use = “required” se especifica la obligatoriedad del valor.
- Restricciones de clave: Ejemplo la definición de nro\_cuenta como clave de cuenta del esquema banco:

```
<xs:key name = “claveCuenta”>  
  <xs:selector xpath = “/banco/cuenta”/>  
  <xs:field xpath = “nro_cuenta”/>  
</xs:key>
```
- Restricción de clave foránea: Ejemplo el titular de una cuenta:

```
<xs:keyref name = “fkCuenta” refer=“claveCuenta”>  
  <xs:selector xpath = “/banco/titular”/>  
  <xs:field xpath = “numero_cuenta”/>  
</xs:keyref>
```



# Consultas y Transformaciones de Datos XML

- Transformación de información de un esquema XML a otro.
- Consultas en datos en formato XML.
- Lenguajes estándares de consulta y transformación:
  - XPath
    - Lenguaje que consiste en expresiones de tipo “path”.
  - XQuery
    - Un lenguaje de query para datos XML que tiene muchas características interesante.
  - XSLT
    - Lenguaje para transformaciones de XML a XML y de XML a HTML.



# XQuery

- XQuery es un lenguaje de propósito general para datos XML.
- Está estandarizado por la World Wide Web Consortium (W3C)  
<http://www.w3.org/TR/2007/REC-xquery-20070123/>
- Xquery fue derivado de Quilt query language, Xquery tienen características de SQL, XQL and XML-QL
- XQuery utiliza expresiones de la siguiente forma  
**for ... let ... where ... order by ...result ...**
- Equivalencia con SQL
  - for** ⇔ SQL **from**
  - where** ⇔ SQL **where**
  - order by** ⇔ SQL **order by**
  - return** ⇔ SQL **select**
  - let** permite variables temporales, esto no tiene equivalencia en in SQL



# Sintaxis FLWOR en XQuery

- La sintaxis de estas expresiones se puede encontrar en la especificación XQuery en:  
<http://www.w3.org/TR/2007/REC-xquery-20070123/#id-flwor-expressions>
- En las cláusulas se utilizan expresiones XPath y variables para almacenar los resultados de las expresiones Xpath.
- Ejemplo de expresión FLWOR en XQuery:
  - En encontrar los números de cuenta de todas las cuentas con saldo > 400, el resultado debe contener cada número de cuenta encerrada con la marca `<numero-cuenta> .. </ numero-cuenta >`  

```
for    $x in /banco-2/cuenta
let    $nroCuenta := $x/@numero-cuenta
where  $x/saldo > 400
return <numero-cuenta> { $nroCuenta } </ numero-cuenta >
```
  - En la cláusula **return** el texto encerrado entre { } son evaluados.
- La expresión del ejemplo anterior se podría escribir sin la cláusula let ni where de la siguiente manera:

```
for $x in /banco-2/cuenta[saldo>400]
return <numero-cuenta> { $x/@numero-cuenta } </numero-cuenta >
```



# Transformaciones XSLT

- Una hoja de estilo almacena opciones de formato para documentos, que generalmente esta separado del documento. Por ejemplo en una hoja de estilo HTML se puede especificar el color y tamaño de la fuente de letra, etc.
- La **XML Stylesheet Language (XSL)** fue originalmente diseñado para generar código HTML a partir de XML
- XSLT es un lenguaje de transformación de propósito general, puede transformar XML a XML, y XML a HTML
- Las transformaciones XSLT son expresadas utilizando reglas llamadas **templates**. Los **templates** combinan selecciones usando XPath con construcción de resultados.
- Especificación disponible en <http://www.w3.org/TR/xslt20/>



# Templates XSLT

- Ejemplo de un template XSLT que hace un “macheo” y un select

```
<xsl:template match="/banco-2/cliente">  
  <xsl:value-of select="nombre_cliente"/>  
</xsl:template>
```

El atributo “match” de xsl:template especifica un patrón en sintaxis XPath.

- Los elementos en el documento XML que machean con el patrón son procesadas por la acción definida en el elemento **xsl:template**.
  - **xsl:value-of select** (salidas) de los valores especificados (en el ejemplo, nombre-cliente)
- Para los elementos que no “machean” con ningún template
  - Sus atributos y texto es generado en la salida.



# JSON

- JSON (JavaScript Object Notation - Notación de Objetos de JavaScript) es un formato liviano de intercambio de datos.
- Es muy simple de interpretar.
- JSON es un formato de texto que es completamente independiente del lenguaje de programación, pero utiliza convenciones que son muy conocidas por programadores de lenguajes C, Java, JavaScript, Python, y muchos otros.
- JSON es un lenguaje ideal para el intercambio de datos, es el lenguaje que esta reemplazando a XML.
- Es el formato de datos utilizados por APIs REST, que está reemplazando a SOAP en la implementación de Web Service.
- Las Bases de datos NoSQL orientadas a documentos utilizan el formato JSON para almacenar los datos, por ejemplo MongoDB.



# Ejemplo de objeto JSON

```
{
  "nombre": "Carlos",
  "apellido": "Perez",
  "edad": 45,
  "direccion": {
    "calle": "San Martin",
    "ciudad": "Rio Cuarto",
    "codigoPostal": 5800
  },
  "telefonos": [
    {
      "tipo": "fijo",
      "numero": "358-45458412"
    },
    {
      "tipo": "movil",
      "numero": "351-468213744"
    }
  ]
}
```

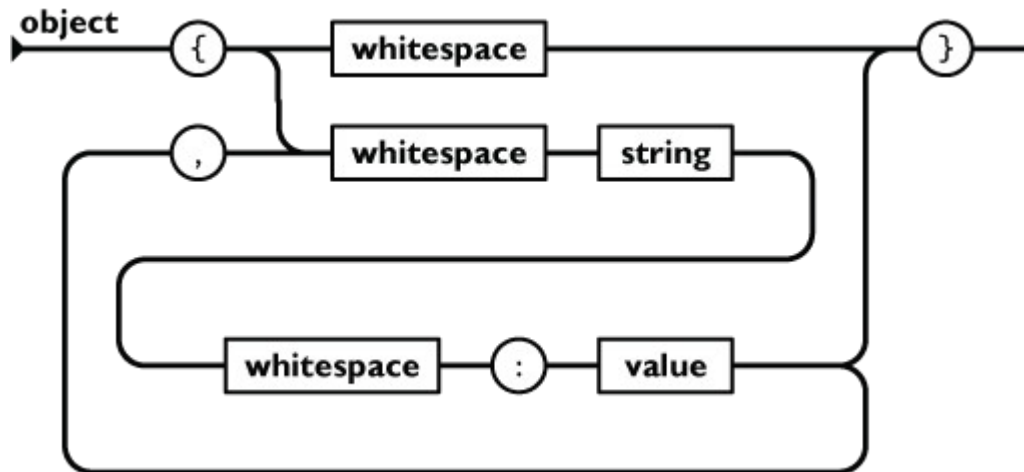


# Estructura de Objetos JSON

JSON está constituido por dos estructuras:

- Una colección de pares de nombre/valor (Objeto).
- Una lista ordenada de valores. En la mayoría de los lenguajes, esto se implementa como arreglos, vectores, listas o secuencias.

Un objeto es un conjunto desordenado de pares nombre/valor y se construyen como indica la figura:

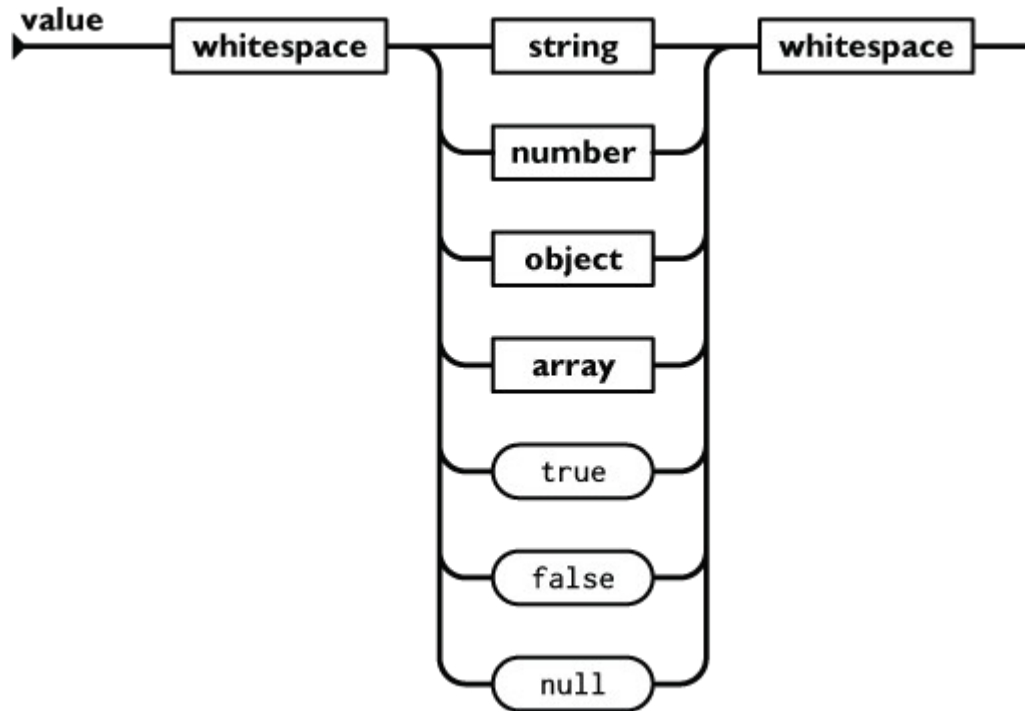


\* Imagen extraída de <https://www.json.org/json-es.html>



# Valores JSON

- Un valor puede ser una cadena de caracteres con comillas dobles, o un número, o true o false o null, o un objeto o un arreglo. Estas estructuras pueden anidarse.



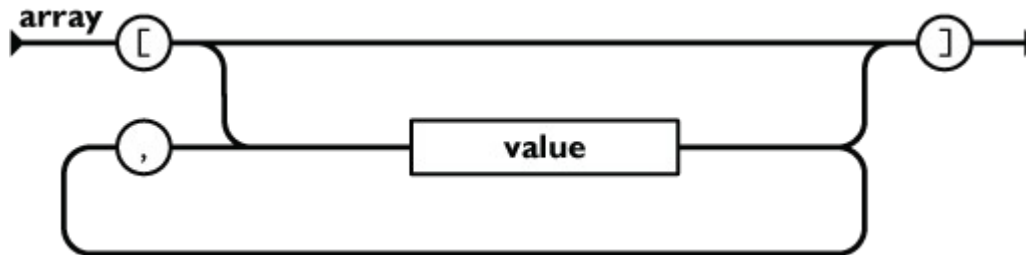
\* Imagen extraída de <https://www.json.org/json-es.html>

\*\* Gramática completa en <https://www.crockford.com/mckeeman.html>



# Arreglos JSON

- Un arreglo es una colección de valores.



\* Imagen extraídas de <https://www.json.org/json-es.html>



# Lenguaje de Consultas para Json

Aún no hay un estándar, pero hay varias propuestas de lenguajes de query para Json. Por ejemplo, la especificación Xquery 3.1 de W3C, extiende el Xquery (XML) a JSON.

Algunos lenguajes de consulta para JSON (disponibles en librerías para diferentes lenguajes) :

- JSONIq (basado en XQuery)
- JSONPath (similar a Xpath)  
(<https://goessner.net/articles/JsonPath/>)
- JSON Query (Tipo de XPath)
- Cada lenguaje de programación tiene su propio lenguaje para consultar documentos JSON, aunque son bastante similares.



# Java Json Path

## Operadores

| Operador                | Descripción                                                                         |
|-------------------------|-------------------------------------------------------------------------------------|
| \$                      | El elemento raíz a consultar. Con esto se inicia todas las path expressions.        |
| @                       | El nodo actual está siendo procesado por un predicado de filtro.                    |
| *                       | Comodín. Disponible en cualquier lugar que se requiera de un nombre o un número.    |
| ..                      | Análisis en profundidad. Disponible en cualquier lugar donde se requiere un nombre. |
| .<name>                 | Acceso a un hijo.                                                                   |
| ['<name>' (, '<name>')] | Hijo/hijos anotados entre corchetes.                                                |
| [<number> (, <number>)] | Uno o mas indices de arreglo.                                                       |
| [start:end]             | Rango de indices de Arreglo.                                                        |
| [?(<expression>)]       | Expresión de filtro. La expresión debe evaluar a un valor booleano.                 |



# Java Json Path (cont.)

## Funciones

| Función  | Descripción                                       | Tipo del retorno |
|----------|---------------------------------------------------|------------------|
| min()    | Retorna el mínimo valor de un arreglo de números. | Double           |
| max()    | Retorna el máximo valor de un arreglo de números. | Double           |
| avg()    | Devuelve el promedio de un arreglo de números.    | Double           |
| length() | Retorna la longitud de un arreglo                 | Integer          |

# Java Json Path (cont.)

## Operadores de Filtro

| Operador | Descripción                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------|
| ==       | igualdad                                                                                                          |
| !=       | distinto                                                                                                          |
| <        | menor                                                                                                             |
| <=       | Menor o igual                                                                                                     |
| >        | mayor                                                                                                             |
| >=       | Mayor o igual                                                                                                     |
| =~       | Operando de la izquierda “machea” con la expresión regular de la derecha, por ejemplo [? (@.nombre =~ /foo.*?/i)] |
| in       | Operando de la izquierda esta contenido en el conjunto de la derecha. Por ejemplo [? (@.tamano in ['S', 'M'])]    |
| nin      | Negación del operador in                                                                                          |
| subsetof | Operando de la izquierda es un subconjunto del conjunto de la derecha [? (@.tamanos subsetof ['S', 'M', 'L'])]    |
| anyof    | Operando de la izquierda tiene intersección con el conjunto d ella izquierda [? (@.tamanos anyof ['M', 'L'])]     |
| noneof   | Negación del operador anyof.                                                                                      |
| size     | Tamaño del arreglo.                                                                                               |
| empty    | El arreglo debe estar vacío                                                                                       |

# Dado el Siguiente JSON

```
{
  "store": {
    "book": [
      {
        "category": "reference",
        "author": "Nigel Rees",
        "title": "Sayings of the Century",
        "price": 8.95
      },
      {
        "category": "fiction",
        "author": "Evelyn Waugh",
        "title": "Sword of Honour",
        "price": 12.99
      },
      {
        "category": "fiction",
        "author": "Herman Melville",
        "title": "Moby Dick",
        "isbn": "0-553-21311-3",
        "price": 8.99
      },
      {
        "category": "fiction",
        "author": "J. R. R. Tolkien",
        "title": "The Lord of the Rings",
        "isbn": "0-395-19395-8",
        "price": 22.99
      }
    ],
    "bicycle": {
      "color": "red",
      "price": 19.95
    }
  },
  "expensive": 10
}
```



# Algunos ejemplos...

| JsonPath                                                | Result                                                                      |
|---------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>\$.store.book[*].author</code>                    | Autores de todos los libros                                                 |
| <code>\$..author</code>                                 | Todos los autores                                                           |
| <code>\$.store.*</code>                                 | Todas la cosas almacenadas, libros y bicicletas                             |
| <code>\$.store..price</code>                            | Precio de todos lo que esta almacenado                                      |
| <code>\$..book[2]</code>                                | El tercer libro                                                             |
| <code>\$..book[-2]</code>                               | Del segundo al ultimo libro                                                 |
| <code>\$..book[0,1]</code>                              | Los primeros dos libros                                                     |
| <code>\$..book[:2]</code>                               | Todos los libros desde el indice 0 (inclusive) hasta el 2 (excluido)        |
| <code>\$..book[1:2]</code>                              | Todos los libros desde el indice 1 (inclusive) hasta el indice 2 (excluido) |
| <code>\$..book[-2:]</code>                              | Últimos dos libros                                                          |
| <code>\$..book[2:]</code>                               | Libro desde el 2 al ultimo                                                  |
| <code>\$..book[?(@.isbn)]</code>                        | Todos los libroscon un ISBN                                                 |
| <code>\$.store.book[?(@.price &lt; 10)]</code>          | Todos los libros que valen menos de 10                                      |
| <code>\$..book[?(@.price &lt;= \$['expensive'])]</code> | Todos los libros baratos                                                    |
| <code>\$..book[?(@.author =~ /. * REES/i)]</code>       | Comando like SQL(ignore case) . * equivale a % en SQL                       |
| <code>\$..*</code>                                      | Devuelve todo                                                               |
| <code>\$..book.length()</code>                          | Cantidad de libros                                                          |

\* Ejemplo extraído de <https://www.json.org/json-es.html>

# Ejemplo Java JsonPath (Jayway)

```
package jsonPath;

import java.io.File;
import java.io.IOException;

import com.jayway.jsonpath.*;
import com.jayway.jsonpath.internal.filter.ValueNode.JsonNode;

public class pruebaJson {

    public static void main(String[] args) {

        String jsonString = "/tmp/json.txt";
        File jsonFile = new File(jsonString);

        String jsonExp = "$.store.book[?(@.price > 10)]";

        try {
            System.out.println("'" + JsonPath.read(jsonFile, jsonExp));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



# Bibliografía Principal

- [SILV 10] Database System Concepts. Sixth Edition, Edition Silberschatz, Korth, Sudarshan. McGraw Hill Company, 2010.
- [SILV 06] Fundamentos de Base de Datos. Quinta Edición, Edition Silberschatz, Korth, Sudarshan. McGraw Hill Company, 2006.
- [JSON] Introducción a JSON, JSON.org, <https://www.json.org/json-es.html>
- [Jayway JsonPath] A Java DSL for reading JSON documents. <https://github.com/json-path/JsonPath>

